

Arquitectura Hardware para la Implementación de la Transformada Discreta Wavelet 2D

Norma Ximena Ríos, Álvaro Bernal

ximenarc@hotmail.com, alvaro.bernal@correounivalle.edu.co

Universidad del Valle

Resumen—El artículo presenta una arquitectura hardware que desarrolla la transformada Wavelet en dos dimensiones sobre una FPGA, en el diseño se buscó un balance entre número de celdas lógicas requeridas y la velocidad de procesamiento. El artículo inicia con una revisión de trabajos previos, después se presentan los fundamentos teóricos de la transformación, posteriormente se presenta la arquitectura propuesta seguida por un análisis comparativo. El sistema se implementó en la FPGA Cyclone II EP2C35F672C6 de Altera utilizando un diseño soportado en el sistema Nios II.

Palabras clave—Transformada Wavelet, Arquitecturas Hardware, Procesador Nios, FPGA.

I. INTRODUCCIÓN

La representación de señales por descomposición no es nueva, aproximadamente hace 200 años Joseph Fourier planteó la representación de funciones mediante la superposición de senos y cosenos, la idea ha evolucionado con el tiempo y las investigaciones más recientes nos llevan a otro tipo de transformaciones entre las cuales encontramos las Wavelets. Las Wavelets son familias de funciones que se emplean para el análisis de otras funciones, dado que permiten representar una señal como una descomposición de señales más sencillas, actualmente versiones discretas de la transformada Wavelet en dos dimensiones (2D-DWT) están siendo utilizadas en importantes aplicaciones de procesamiento de imágenes digitales tales como: sistemas de compresión, eliminación de ruido,

sistemas de radar, sistemas electrocardiográficos, entre otras.

En la actualidad se requieren sistemas que sean cada vez más pequeños y más rápidos, es por esto que se hace necesario plantear una arquitectura hardware para el cálculo de la 2D-DWT que tenga alta eficiencia con un bajo costo hardware; Chen et al, en [1] proponen una arquitectura de procesamiento paralelo que calcula la 2D-DWT mediante una adaptación del Algoritmo Piramidal Recursivo (RPA) [2], la idea general del RPA es replantear el orden en el cual se calculan los coeficientes de la transformada de forma tal que pueda iniciarse el cálculo del siguiente nivel sin completar el cálculo de los coeficientes del nivel anterior, Chen presenta una adaptación para transformaciones en dos dimensiones y en lugar de organizar la transformación realizando el cálculo píxel a píxel se planifica por filas, por consiguiente se deben calcular en paralelo los coeficientes de toda una fila, el esfuerzo requerido para mantener el rastro de los últimos coeficientes calculados aumenta la complejidad del controlador convirtiéndose en la principal desventaja. Vishwanath et al, en [3] presentan dos arquitecturas, la primera consiste esencialmente en un módulo de transformación en una dimensión que es utilizado repetidas veces para calcular la 2D-DWT, la ventaja de la arquitectura es su simplicidad pero requiere demasiadas celdas de memoria lo que la hace poco conveniente para la implementación en un chip, otra desventaja es la latencia necesaria para que el primer dato de salida sea generado; la segunda arquitectura propuesta por Vishwanath et al, está compuesta por un filtro sistólico que maneja el filtrado en la dirección horizontal, un filtro paralelo

Artículo presentado en Abril de 2012, como resultado del trabajo de investigación en sistemas de compresión desarrollado en la Universidad del Valle.

N.X. Ríos, docente de la Facultad de Ingeniería de la Institución Universitaria Antonio José Camacho. (e-mail: xrios@admon.uniajc.edu.co)

A. Bernal, docente de la Escuela de Ingeniería Eléctrica y Electrónica de la Universidad del Valle. (e-mail: alvaro.bernal@correounivalle.edu.co).

para manejar el filtrado en la dirección vertical y una unidad de almacenamiento, donde dos filas son procesadas en el filtro sistólico en el orden del esquema del RPA, a su vez el filtro paralelo calcula 4 filas que constituyen cuatro salidas del primer nivel y una de ellas es llevada al filtro sistólico para continuar con el procesamiento, de esta forma se logra mejorar el rendimiento con respecto a la primera arquitectura pero se incrementa el área requerida para su implementación. Colom et al, en [4][5] presentan una arquitectura que trabaja con filtros 2D no-separables que utiliza una estructura paralela denominada *even-odd*, la arquitectura es recurrente; existe una unidad de filtros para realizar el cálculo del primer nivel y está continuamente procesando; otra unidad de filtros se encarga del cálculo del resto de niveles e inicia cuando la primera unidad ha generado las primeras cuatro filas y sigue calculando cada vez que la primera genera dos nuevas líneas, los periodos de tiempo intermedios son aprovechados para realizar la recurrencia, se utilizan dos unidades de almacenamiento que actúan de enlace entre niveles, la arquitectura utiliza unidades de control distribuidas que facilitan la escalabilidad, existe un control al interior de la unidad de filtros y un control para sincronizar el funcionamiento entre dos niveles consecutivos, al incrementar el número de niveles se debe replicar el número de unidades de control. Sheu et al, en [6] proponen una arquitectura que consta de dos módulos de filtros horizontales para calcular los coeficientes a lo largo de las filas y dos módulos verticales para calcular a lo largo de las columnas, cada módulo se compone de un filtro de paso alto y un filtro de paso bajo, el filtro horizontal procesa las filas y almacena el resultado en la primera unidad de almacenamiento, después este resultado es procesado en el filtro vertical, la salida de la parte baja del filtro es cargada en el filtro horizontal y se itera el proceso, los módulos de filtros se fundamentan en una metodología de reuso de los datos de entrada con una estructura paralelo-*pipelined*. Chakrabarti et al, en [7] presentan dos arquitecturas, la primera consta de dos unidades de memoria y cuatro unidades de filtros paralelas compuestas por un filtro de paso alto y uno de paso bajo, los dos primeros filtros

calculan a lo largo de las filas, su salida es almacenada en la primera unidad de memoria donde los datos son leídos por columnas por los siguientes dos filtros y se calculan los coeficientes a lo largo de las columnas, similarmente las salidas de estos filtros son almacenadas en la segunda unidad de memoria por columnas y leídas en filas por el segundo filtro, en la publicación se presentan dos algoritmos de planificación del flujo de datos que pueden ser utilizados sobre esta arquitectura, los autores manifiestan que debido a que las unidades de filtros son usadas recursivamente para calcular dos subimágenes, se genera un retardo de N ciclos lo cual puede ser inaceptable para algunas aplicaciones; la segunda arquitectura propuesta por Chakrabarti et al, es una versión modificada de la anterior donde se busca disminuir el retardo generado por la recursividad, en esta propuesta se incrementan dos unidades de filtros para producir al mismo tiempo la salida de todas las subimágenes de un mismo nivel, con lo cual se logra disminuir el tamaño de las unidades de almacenamiento y el retardo.

En este artículo se propone una arquitectura con un ruteo simple y una unidad de control de complejidad moderada que disminuye el tiempo necesario para calcular la transformada discreta Wavelet en dos dimensiones.

II. TRANSFORMADA WAVELET DISCRETA EN DOS DIMENSIONES (2D-DWT)

El análisis mediante Wavelet se fundamenta en la dilatación y traslación de una función de escala y la función Wavelet asociada para obtener la representación de una señal a diferentes resoluciones; uno de los grandes avances en el análisis Wavelet de señales digitales fue su implementación usando filtros, los cuales fueron formados utilizando los coeficientes de las funciones de escalamiento y Wavelet [8], el filtro de paso bajo (h) esta asociado a la función de escalamiento y la señal obtenida a su salida es una versión suavizada de baja resolución de la señal original, el filtro de paso alto (g) esta asociado a la función Wavelet y la señal obtenida a su salida contiene los detalles de la señal; para el caso de las

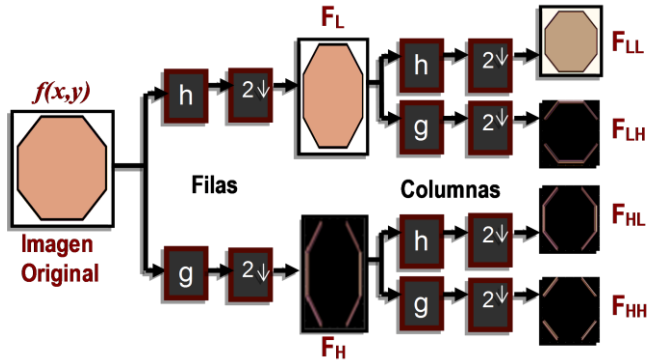


Fig. 1. Diagrama de bloques del banco de filtros utilizado para calcular la 2D-DWT

imágenes trabajaremos en dos dimensiones extendiendo la transformada unidimensional a funciones bidimensionales; la figura 1 ilustra el desarrollo de la transformada Wavelet de una imagen utilizando filtros unidimensionales, primero se desarrolla la transformación Wavelet sobre cada una de las filas de la imagen, lo que genera dos imágenes intermedias que representan la aproximación F_L y el detalle F_H sobre el eje x , posteriormente se realiza la transformación sobre cada columna de las imágenes intermedias, como resultado se obtiene una versión suavizada de la imagen o promedio F_{LL} y tres subimágenes con los detalles F_{LH} , F_{HL} , F_{HH} . F_{LH} enfatiza las característica horizontales, F_{HL} las verticales y F_{HH} las diagonales [9]. La transformación puede ser definida de acuerdo con las expresiones [4]:

$$F_{LL}(x,y) = \sum_{k_1 k_2} h(k_1)h(k_2)f(2x-k_1, 2y-k_2) \quad (1)$$

$$F_{LH}(x,y) = \sum_{k_1 k_2} h(k_1)g(k_2)f(2x-k_1, 2y-k_2) \quad (2)$$

$$F_{HL}(x,y) = \sum_{k_1 k_2} g(k_1)h(k_2)f(2x-k_1, 2y-k_2) \quad (3)$$

$$F_{HH}(x,y) = \sum_{k_1 k_2} g(k_1)g(k_2)f(2x-k_1, 2y-k_2) \quad (4)$$

Donde (x,y) representa las coordenadas de las imágenes, h y g son los filtros de paso alto y de paso bajo, f es la imagen de entrada y k representa el tamaño de los filtros

El proceso se puede iterar para niveles superiores, asumiendo la imagen promedio F_{LL} como entrada del siguiente nivel; la figura 2 ilustra la transformación para dos niveles.

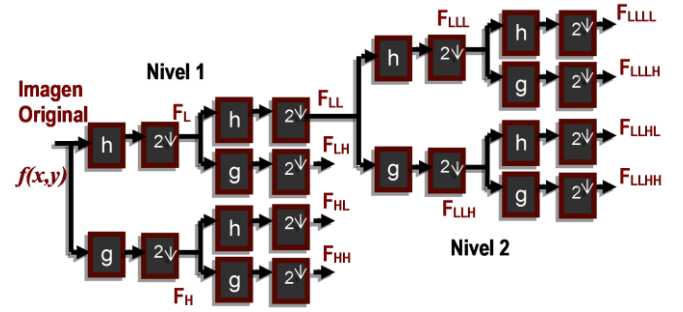


Fig. 2. Diagrama de bloques del banco de filtros utilizado para calcular dos niveles de la 2D-DWT

III. ARQUITECTURA PROPUESTA PARA LA DWT-2D

La figura 3 muestra el diagrama de bloques de la arquitectura que se propone para la implementación de la 2D-DWT en hardware; la arquitectura está compuesta por tres unidades de memoria (UM1, UM2, UM3), una unidad de control y tres unidades de filtros paralelos (UF1, UF2, UF3) compuestas por un filtro de paso alto y uno de paso bajo.

La unidad de control se encarga de la planificación del flujo de datos de la siguiente manera: la UF1 procesa la imagen de entrada a lo largo de las filas y genera las imágenes intermedias F_H y F_L , estas subimágenes son almacenadas en la UM1 y UM2 para ser cargadas en UF2 y UF3 donde se procesan las imágenes por columnas, en UF2 se generan los coeficientes de las subimágenes F_{HL} , F_{HH} y en UF3 se generan F_{LL} , F_{LH} , la salida de la componente baja de UF3 es almacenada en UM3 para posteriormente ser cargada en UF1 donde se itera el proceso para el cálculo del siguiente nivel.

El diseño es implementado sobre la tarjeta de desarrollo DE2 [10], esta tarjeta cuenta con una FPGA Cyclone II EP2C35F672C6 [11] y con

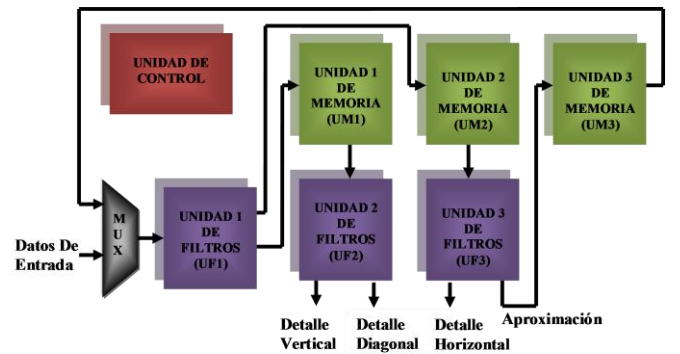


Fig. 3. Arquitectura Paralela Por Nivel

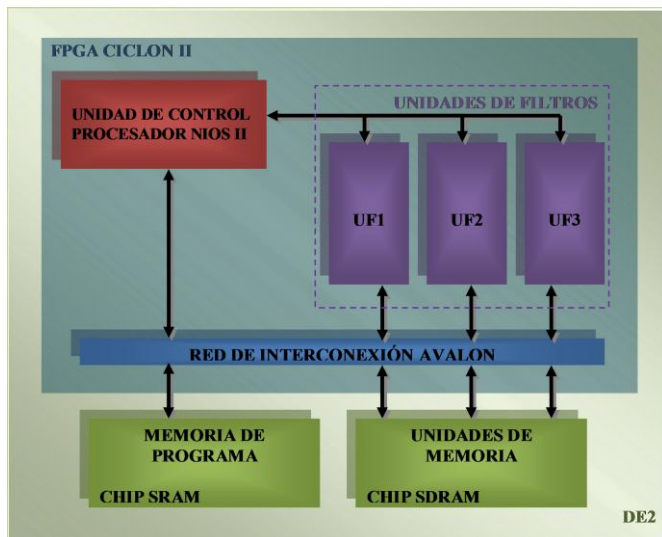


Fig. 4. Diagrama de Bloques del Sistema Implementado en la Tarjeta de Desarrollo DE2

unidades de memoria e interfaces de entrada y salida que la convierten en una plataforma ideal para prototipado de aplicaciones en multimedia y redes. La figura 4 muestra el diagrama de bloques del sistema implementado sobre la DE2.

IV. UNIDAD DE MEMORIA

Como se menciona en la sección anterior las unidades de memoria son utilizadas para almacenar los datos resultado de la transformación; la UM1 y la UM2 almacenan los resultados de la transformación a lo largo de las filas (F_L , F_H) para lo cual se requiere de $N \times N/2$ celdas de memoria para almacenar el resultado de procesar una imagen de tamaño $N \times N$, la UM3 almacena los resultados de la componente baja frecuencia de la transformación por columnas para lo cual se requiere de $N/2 \times N/2$ celdas, por lo tanto el total de memoria requerida

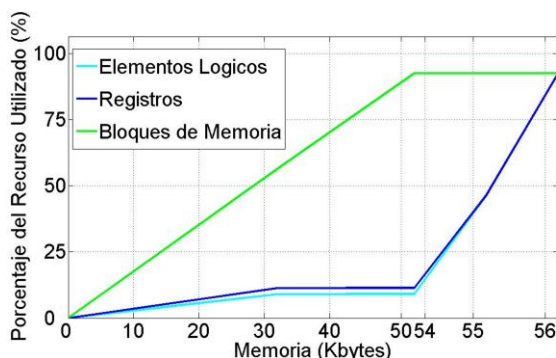


Fig. 5. Relación Entre el Tamaño de la Memoria y el Consumo de Elementos Lógicos y Registros

por la arquitectura propuesta está determinado por:

$$T_{MEM} = N \cdot \frac{N}{2} + N \cdot \frac{N}{2} + \frac{N}{2} \cdot \frac{N}{2} = \frac{5}{4} N^2 \quad (5)$$

La FPGA Cyclone II EP2C35F672C6 cuenta con una estructura de memoria interna organizada en 3 columnas que contienen un total de 105 bloques que proveen una capacidad de almacenamiento al interior de la FPGA de 483840bits y una velocidad máxima de operación de 250MHz [11], lo cual solo permitiría procesar imágenes con N inferior a 220 píxeles; aunque se puede ampliar la capacidad de almacenamiento interno utilizando los bloques de arreglos lógicos para almacenar datos, no es recomendable puesto que sólo se aumentan 2047bytes utilizando todos los recursos de la FPGA, esto se evidencia en la figura 5 donde se muestra la relación entre el tamaño de la memoria y el consumo de elementos lógicos, registros y bloques de memoria; se nota que una vez se alcanza el límite de almacenamiento de los bloques de memoria se eleva el consumo de elementos lógicos y registros. Para procesar imágenes de mayor tamaño se amplió la capacidad de almacenamiento del sistema utilizando una de las memorias de la tarjeta DE2, se uso la SDRAM IS42S16400 que almacena 8Mbytes de datos, para la interfaz de conexión Altera ha desarrollado una herramienta llamada SOPC (System on a Programmable Chip) [12] que permite manipular controladores predefinidos y recurre a la red de interconexión AVALON que maneja menos elementos lógicos en la conexión y mejora el desempeño en la velocidad de transmisión. En el diagrama de bloques de la figura 4 se observa que para la implementación de las unidades de memoria se conecta a través de la red AVALON una SDRAM externa a la FPGA.

V. UNIDAD DE FILTROS

La unidad de filtros se fundamenta en una metodología de reuso de los datos de entrada con una estructura paralelo-*pipelined* similar a la propuesta por Sheu et al, en [6] pero el cálculo de los coeficientes se realiza con la metodología *even-odd* propuesta por Colom et al, en [4][5], con la que se logra calcular un coeficiente por ciclo después de 2 ciclos de latencia; el esquema general de la unidad

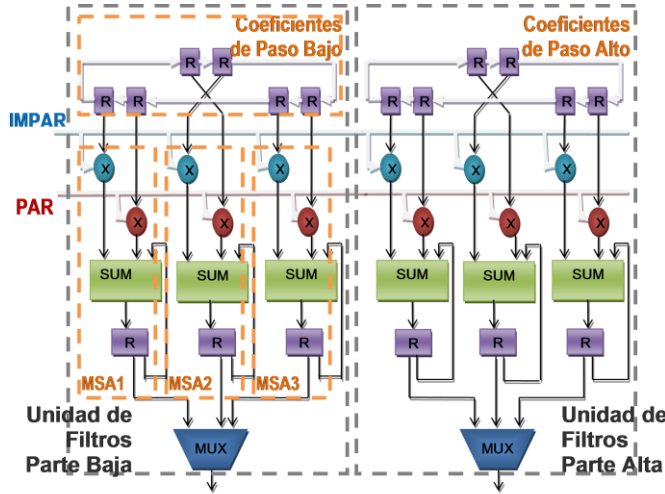


Fig. 6. Unidad de Filtros

de filtros se presenta en la figura 6. Cada unidad tiene dos filtros y cada filtro está compuesto por: un registro de corrimiento que almacenan los coeficientes del filtro y está configurado de manera que en cada ciclo realiza dos desplazamientos, tres unidades de Multiplicador-Sumador-Acumulador (MSA1, MSA2, MSA3) compuestas por dos multiplicadores (uno para los datos pares y otro para los datos impares) un sumador y un registro acumulador, un multiplexor que selecciona cuál de los datos se encuentra listo para ser enviado a la siguiente etapa.

Para ilustrar el funcionamiento de esta unidad se consideraron dos filtros h (paso bajo) y g (paso alto) con seis coeficientes definidos como:

$$h = [h_1, h_2, h_3, h_4, h_5, h_6] \quad (6)$$

$$g = [g_1, g_2, g_3, g_4, g_5, g_6] \quad (7)$$

Sea f una señal con N datos, cuyos valores discretos están definidos así:

$$f = [f_1, f_2, f_3, f_4, \dots, f_N] \quad (8)$$

Al transformar la señal f usando los filtros g y h se obtienen dos señales, una de aproximación (A) y una con los detalles (D), cuyos coeficientes se pueden expresar de la siguiente manera:

$$A = [A_1, A_2, A_3, A_4, \dots, A_{N/2}] \quad (9)$$

$$D = [D_1, D_2, D_3, D_4, \dots, D_{N/2}] \quad (10)$$

Teniendo en cuenta que la transformación

unidimensional [4] esta definida por las siguientes ecuaciones:

$$A[x] = \sum_k h[k] f[2x - k] \quad (11)$$

$$D[x] = \sum_k g[k] f[2x - k] \quad (12)$$

Podemos usar la ecuación 11 para expresar cada coeficiente de A de la siguiente forma:

$$A_1 = f_1 \cdot h_1 + f_2 \cdot h_2 + f_3 \cdot h_3 + f_4 \cdot h_4 + f_5 \cdot h_5 + f_6 \cdot h_6 \quad (13)$$

$$A_2 = f_3 \cdot h_1 + f_4 \cdot h_2 + f_5 \cdot h_3 + f_6 \cdot h_4 + f_7 \cdot h_5 + f_8 \cdot h_6 \quad (14)$$

$$A_3 = f_5 \cdot h_1 + f_6 \cdot h_2 + f_7 \cdot h_3 + f_8 \cdot h_4 + f_9 \cdot h_5 + f_{10} \cdot h_6 \quad (15)$$

$$A_4 = f_7 \cdot h_1 + f_8 \cdot h_2 + f_9 \cdot h_3 + f_{10} \cdot h_4 + f_{11} \cdot h_5 + f_{12} \cdot h_6 \quad (16)$$

$$A_5 = f_9 \cdot h_1 + f_{10} \cdot h_2 + f_{11} \cdot h_3 + f_{12} \cdot h_4 \quad (17)$$

$$A_6 = f_{11} \cdot h_1 + f_{12} \cdot h_2$$

Hasta llegar a los límites de la señal donde:

$$A_{N/2} = f_{N-1} \cdot h_1 + f_N \cdot h_2 \quad (17)$$

La metodología de reuso consiste en organizar el flujo de los datos de entrada de forma que sea posible calcular en paralelo varios de los datos de salida, este proceso se ilustra en la figura 7.

En cada ciclo el registro de coeficientes se desplaza dos espacios; así en el primer ciclo a MSA1 llegan los coeficientes h_1, h_2 , a MSA2 llegan h_3, h_4 y a MSA3 llegan h_5, h_6 ; en el segundo ciclo a MSA1 llegan los coeficientes h_3, h_4 , a MSA2 llegan h_5, h_6 y a MSA3 llegan h_1, h_2 ; en el tercer ciclo a MSA1 llegan los coeficientes h_5, h_6 , a MSA2 llegan h_1, h_2 y a MSA3 llegan h_3, h_4 y el proceso se repite hasta finalizar la transformación. Por otro lado en cada ciclo entran de manera paralela dos datos (par e impar) para ser procesados; en el primer ciclo ingresan f_1, f_2 con estos datos el MSA1 inicia el cálculo de A_1 acumulando $f_1 \cdot h_1 + f_2 \cdot h_2$; en el segundo ciclo ingresan f_3, f_4 , el MSA1 acumula $f_3 \cdot h_3 + f_4 \cdot h_4$ y de manera paralela MSA2 utiliza los mismos datos

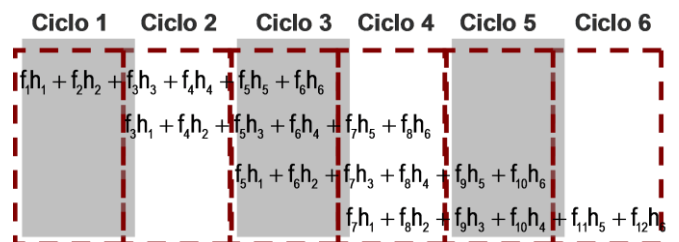


Fig. 7. Flujo de datos por ciclo

de entrada para iniciar el cálculo de A_2 acumulando $f_3.h_1 + f_4.h_2$; en el tercer ciclo ingresan f_5, f_6 , el MSA1 calcula $f_5.h_5 + f_6.h_6$ con lo que finaliza el cálculo de A_1 , de manera paralela MSA2 calcula $f_5.h_3 + f_6.h_4$ y MSA3 inicia el cálculo de A_3 acumulando $f_5.h_1 + f_6.h_2$, en adelante se entregará un dato de salida en cada ciclo y el multiplexor conmuta para transportarlo a la salida; en el cuarto ciclo ingresan f_7, f_8 , el MSA1 inicia el cálculo de A_4 acumulando $f_7.h_1 + f_8.h_2$, el MSA2 calcula $f_7.h_5 + f_8.h_6$ con lo que finaliza el cálculo de A_2 , el MSA3 acumula $f_7.h_3 + f_8.h_4$; en el quinto ciclo ingresan f_9, f_{10} , el MSA1 acumula $f_9.h_3 + f_{10}.h_4$, el MSA2 inicia el cálculo de A_5 y el MSA3 finaliza el cálculo de A_3 acumulando $f_9.h_5 + f_{10}.h_6$; en adelante el proceso se itera hasta llegar al final de la señal de entrada.

Siguiendo la metodología descrita en el párrafo anterior se calculan los componentes de paso alto de la señal de salida en una arquitectura paralela, la única diferencia se encuentra en los datos almacenados en el registro de corrimiento que deben corresponder a los coeficientes del filtro de paso alto.

Note que cada dato de entrada se utiliza en el cálculo parcial de varios de los coeficientes de salida. Al realizar el procesamiento de los pares en paralelo con los impares se obtienen dos datos de salida (uno de paso alto y uno de paso bajo) en cada ciclo de reloj a partir del tercer ciclo. Por lo tanto la transformación de una señal unidimensional de tamaño N se realiza en $N/2+2$ ciclos de reloj, si ampliamos el procesamiento para una señal en dos dimensiones de tamaño $N \times N$ se requieren $(N/2+2)N$ ciclos para procesar todas las filas y $(N/2+2)N/2$ ciclos para procesar las columnas, por lo tanto el total de ciclos requeridos por la arquitectura propuesta para calcular los coeficientes del primer nivel de transformación está determinado por:

$$T_{CICLOS} = \left(\frac{N}{2} + 2 \right) N + \left(\frac{N}{2} + 2 \right) \frac{N}{2} = \frac{3}{4} N^2 + 3N \quad (18)$$

El algoritmo de planificación del flujo al interior de la unidad de filtros puede expresarse de la siguiente manera [14]:

X=1

PARA i=1 HASTA N CON INCREMENTO +2

$$A_X = f_i h_1 + f_{i+1} h_2$$

$$D_X = f_i g_1 + f_{i+1} g_2$$

SI (X>1) ENTONCES

$$A_{X-1} = A_{X-1} + f_i h_3 + f_{i+1} h_4$$

$$D_{X-1} = D_{X-1} + f_i g_3 + f_{i+1} g_4$$

FIN SI

SI (X>2) ENTONCES

$$A_{X-2} = A_{X-2} + f_i h_5 + f_{i+1} h_6$$

$$D_{X-2} = D_{X-2} + f_i g_5 + f_{i+1} g_6$$

FIN SI

X=X+1

FIN PARA

La unidad de filtros se modeló utilizando VHDL y las megafunciones de Altera, el diseño tiene 5 módulos estructurales: Registro de Coeficientes, Multiplicador, Sumador, Registro Acumulador y Multiplexor. Para la implementación de los multiplicadores se utilizó la megafunción de Altera *lpm_mul* que permite utilizar los multiplicadores embebidos optimizando el uso de la FPGA, para el Multiplexor se utilizó la megafunción de Altera *lpm_mux* dado que al sintetizar el diseño descrito en VHDL se requerían 17 elementos lógicos adicionales; el módulo sumador se implementó con la megafunción *parallel_add* puesto que el diseño descrito en VHDL requería de un tiempo de estabilización mayor en 0.577ns, los elementos restantes y las líneas de conexión fueron modelados en VHDL.

Esta unidad fue sintetizada en una FPGA Cyclone II EP2C35F672C6 [11] de Altera utilizando la herramienta de diseño Quartus II versión 6.0 *web edition*, la tabla I muestra los recursos usados en la implementación de la unidad de filtros.

En la tabla I observamos que en la implementación de la unidad de filtros se utiliza el 34% de los multiplicadores embebidos, el 1% de los elementos lógicos y 0% de los bits de memoria, ello evidencia que disponemos de suficiente espacio al

TABLA I
RECURSOS UTILIZADOS EN LA UNIDAD DE FILTROS

Tipo	Recursos Utilizados
Elementos Lógicos	292 / 33216 (<1%)
Registros	219
Bits de Memoria	0 / 483840 (0%)
Multiplicadores Embebidos	24 / 70 (34%)
PLL	0 / 4 (0%)

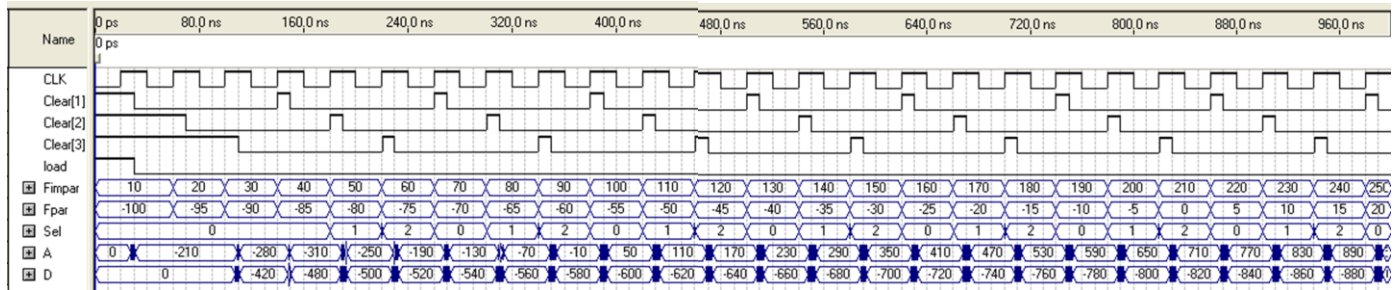


Fig. 8. Simulación Unidad de Filtros

interior de FPGA para aumentar el grado de paralelismo en el diseño y utilizar simultáneamente varias unidades de filtros, lo que permitiría obtener los coeficientes de transformación de niveles superiores en menor tiempo ocupando la totalidad de los multiplicadores disponibles, en aplicaciones donde se utilice la FPGA sólo para la implementación de la etapa de transformación ésta es una opción viable, sin embargo en aplicaciones donde se necesite implementar funciones adicionales en la FPGA es necesario el uso recursivo de las unidades de filtros para el cálculo de los siguientes niveles de transformación.

En la figura 8 se muestran parte de los resultados de la simulación de la unidad de filtros, en el primer ciclo de reloj se realiza la carga de los registros de coeficientes con los valores de los filtros utilizando la señal de control de carga (Load) y se limpian los registros acumuladores usando la señal de limpieza (Clear), en el segundo ciclo se realizan operaciones parciales; a partir del tercer ciclo se obtienen dos coeficientes de salida en cada ciclo para lo cual se conmutan los multiplexores a través de su señal de selección (Sel), es necesario cada tres ciclos generar señales de limpieza escaladas para borrar los registros acumuladores, los datos pares están representados por la señal llamada Fpar y los impares por Fimpar, los coeficientes de alta frecuencia de la transformación por la señal de salida A y los de baja frecuencia por D; para la validación de los datos obtenidos se modeló el sistema en Matlab y se compararon los resultados, los datos obtenidos de la transformación procesada en Matlab y los obtenidos de la simulación en Quartus concuerdan totalmente (error 0%), lo cual se ajusta a lo esperado puesto que en esta etapa los datos aun no han sido cuantificados

VI. UNIDAD DE CONTROL

Para la unidad de control de la arquitectura se utiliza un diseño soportado en el sistema Nios II [13], Nios II es un procesador definido en un lenguaje de descripción hardware que puede ser implementado en FPGAs de Altera usando la herramienta de síntesis Quartus II en conjunción con el SOPC Builder, existen tres versiones del procesador: la económica Nios/e, la estándar Nios/s y la rápida Nios/f. La versión económica tiene el

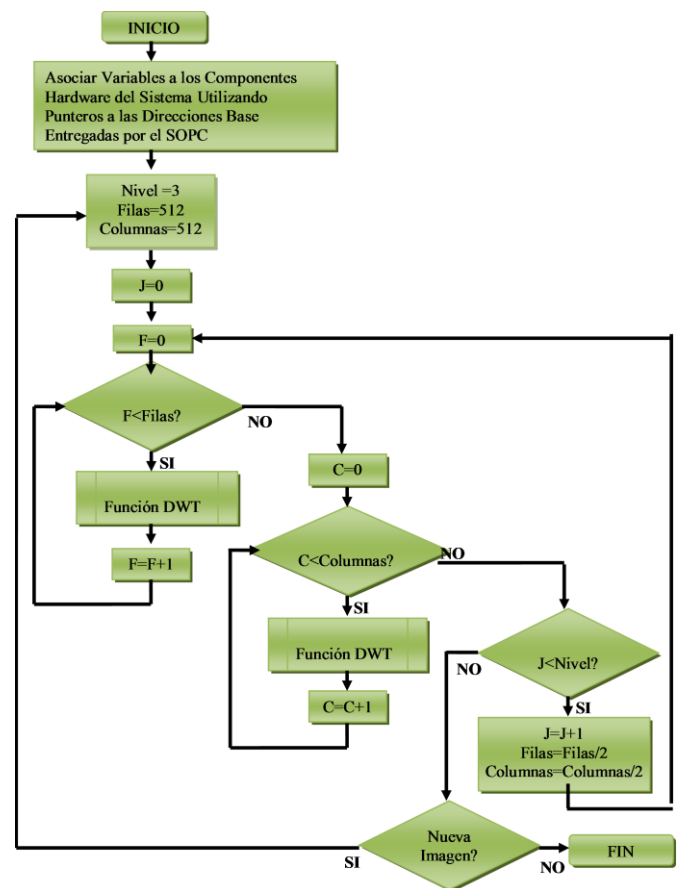


Fig. 9. Diagrama de Flujo del Programa de la Unidad de Control

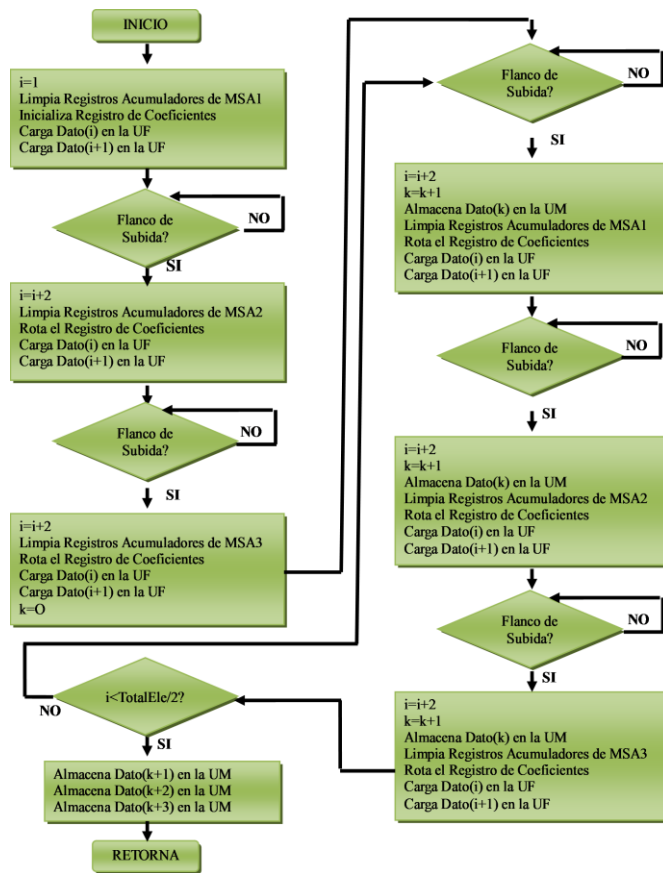


Fig. 10. Diagrama de Flujo de la Función para el Cálculo de la Transformada en una Dimensión

núcleo de menor tamaño pero no maneja memoria cache ni hardware especializado para desarrollar operaciones aritméticas, para realizar el control de la arquitectura se escogió la versión económica puesto que las operaciones que se requieren para

controlar la arquitectura no incluyen operaciones complejas.

Las instrucciones de control se desarrollan en C++ utilizando el entorno de desarrollo integrado NIOS II IDE que permite compilar, depurar y cargar fuentes en C/C++ sobre el sistema desarrollado. Para almacenar el programa a ejecutar es preciso destinar un espacio en memoria, la tarjeta DE2 posee un chip de memoria SRAM que almacena 512Kbytes y se incluye en el sistema como memoria de programa (ver figura 4), al igual que la SDRAM la SRAM se conecta a la red AVALON utilizando el *SOPC Builder*. El diagrama de flujo del programa principal se presenta en la figura 9 y el diagrama de flujo de la función para el cálculo de la transformada en una dimensión se presenta en la figura 10.

VII. RENDIMIENTO Y COMPARACIONES DE ARQUITECTURAS

En las arquitecturas implementadas sobre FPGAs se debe tener en cuenta que éstas contienen un número limitado de celdas lógicas, por lo que es necesario que el diseño se oriente hacia la optimización de este recurso, considerando que se requiere un alto número de celdas lógicas para la implementación de elementos de almacenamiento es conveniente utilizar arquitecturas con un menor requerimiento de memoria, de igual forma la

TABLA II
RENDIMIENTO Y COMPARACIÓN DE ARQUITECTURAS

Arquitectura	Requerimiento de Almacenamiento	Tiempo de Cálculo	Ruteo	Control
Arquitectura Propuesta en Este Trabajo	$\frac{5}{4}N^2$	$\frac{3}{4}N^2 + 3N$	simple	moderado
Arquitectura de Procesamiento Paralelo por Filas Chen et al	$(K+1)NJ$	$N^2 + N$	complejo	complejo
Arquitectura de Aproximación Directa Vishwanath et al	N^2	$4N^2$	simple	simple
Arquitectura Sistólica-Paralela Vishwanath et al	$2NK$	$N^2 + N$	complejo	complejo
Arquitectura Recurrente para Tres Niveles Colom et al	$6N$	$N^2 + N$	moderada	moderado
Arquitectura Recurrente para Tres Niveles Sheu et al	$N/2 + N/4$	$N^2 + N$	moderada	moderado
Arquitectura 1 Paralela Chakrabarti et al	$\approx N\left(3/2 - 2^{1-J}\right) + KN\left(2 - 2^{1-J}\right)$	$\approx N^2$	moderada	moderado
Arquitectura 2 Paralela Chakrabarti et al	$\approx KN\left(1 - 2^{-J}\right) + N\left(1 - 2^{1-J}\right)$	$\approx N^2$	moderada	moderado

complejidad del ruteo también lleva a un alto consumo de celdas lógicas pues se requieren más elementos para interconectar el sistema siendo conveniente una red de ruteo poco compleja; otro factor primordial es el tiempo necesario para el cálculo de la transformación, éste debe reducirse al mínimo para procesar grandes cantidades de imágenes a tasas satisfactorias; de igual forma es conveniente reducir la complejidad del sistema de control para que la arquitectura sea fácilmente escalable y programable. El logro de todos estos objetivos es particularmente difícil en un FPGA ya que área y velocidad son inversamente proporcionales y la satisfacción de un requerimiento significa afectar de manera sensible el otro, por ejemplo, se puede aumentar la velocidad con un alto grado de paralelismo lo que implica un aumento en el área requerida. La tabla II relaciona estas características para las arquitecturas consultadas y la arquitectura propuesta en este trabajo, en ella podemos observar que la arquitectura propuesta en este trabajo presenta una disminución en el tiempo de cálculo en comparación con las arquitecturas consultadas, en el caso mas notable es mejorada en aproximadamente $3N^2$ ciclos y en el caso menos notable aproximadamente en $0.25N^2$ ciclos, la mejora no complica la implementación pues se conserva un ruteo simple y una unidad de control de complejidad moderada. Si bien en nuestra propuesta se requieren más celdas de almacenamiento este percance se supera al utilizar una memoria externa para el almacenamiento de los datos.

VIII. CONCLUSIONES

La arquitectura hardware presentada para desarrollar la transformada discreta Wavelet en dos dimensiones provee un desempeño eficiente en área y velocidad de cálculo, la arquitectura utiliza $3/4N^2 + 2N$ ciclos para transformar una imagen de tamaño $N \times N$ logrando una mejora frente a las arquitecturas desarrolladas en trabajos previos, además mantiene un control y un ruteo de complejidad moderada, esta arquitectura está adaptada a un sistema de transformación de imágenes sin embargo puede ser usada en cualquier

aplicación que requiera análisis frecuencial en subbandas de diferentes señales.

REFERENCIAS

- [1] C. Chen, Z. Yang, T. Wang, and L. Chen, "A programmable VLSI architecture for 2-D discrete wavelet transform," *Proceedings, IEEE International Symposium on Circuits and Systems*, pp. 619-622, Mayo 2000.
- [2] M. Vishwanath, "The Recursive Pyramid Algorithm for the discrete wavelet transform," *IEEE Transactions on Signal Processing*, vol. 42, no. 3, pp. 673-676, Marzo 1994.
- [3] M. Vishwanath, R. M. Owens, and M. J. Irwin, "VLSI architectures for the discrete wavelet transform," *IEEE Transactions on Circuits and Systems - II*, vol. 42, no. 3, pp. 305-316, Mayo 1995.
- [4] R. J. Colom, R. Gadea, A. Sebastiá, M. Martinez, V. Herrero, V. Arnau, "Transformada Discreta Wavelet 2-D para Procesamiento de Vídeo en Tiempo Real," presentado en XII Jornadas de Paralelismo. Valencia, España, 2001, disponible online en http://www.uv.es/varnau/jor_pal_2001.pdf consultado en octubre 2010.
- [5] R. J. Colom, R. Gadea, A. Sebastiá, M. Martinez, F. Ballester, V. Herrero, "Implementación de la Transformada Wavelet Discreta 2D con filtros no separables," presentado en I Jornadas sobre Computación Reconfigurable y Aplicaciones. Alicante, España, 2001.
- [6] M. Sheu, M. Shieh, S. Liu, "A VLSI Architecture Design With Lower Hardware Cost and Less Memory for separable 2-D Discrete Wavelet Transform," *IEEE Circuits and Systems, Proceedings of International*
- [7] C. Chakrabarti, C. Mumford, "Efficient Realizations of Encoders and Decoder Based on the 2-D Discrete Wavelet Transform," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 7, no. 3, pp. 289-298, Septiembre 1999.
- [8] S. Grgic, M. Grgic, "Performance Analysis of Image Compression Using Wavelets," *IEEE Transactions on Industrial Electronics*, vol. 48, no. 3, pp. 682-695, Junio 2001.
- [9] M.L. Hilton, B.D. Jawerth, A. Sengupta, "Compressing still and moving images with wavelets," *Multimedia Systems*, vol. 2, no. 3, pp. 218-227, Abril 1994.
- [10] DE2 User Manual, Altera Corporation, disponible online en <ftp://ftp.altera.com/up/pub/Webdocs> consultado en junio 2010
- [11] Cyclone II Device Handbook, Altera Corporation, Vol. 1, 2008, disponible online en <http://www.altera.com/literature/lit-cyc2.jsp> consultado en junio 2011.
- [12] User Guide SOPC Builder, Altera Corporation, disponible online en http://www.altera.com/literature/lit-nio2.jsp#related_documentation consultado en junio 2011.
- [13] Nios II Processor Reference Handbook, Altera Corporation, versión 11.0, 2011, disponible online en

http://www.altera.com/literature/lit-nio2.jsp#related_documentation consultado en junio 2011.

- [14] X. Ríos, A. Bernal, “Implementación de un Sistema de Compresión en el Dominio Wavelet sobre una FPGA usando el Procesador Nios II”, II Congreso Internacional de Instrumentación, Control y Telecomunicaciones. Tunja, Colombia, octubre , 2011



Norma Ximena Ríos, en 2005 obtuvo su título de Ingeniera Electrónica en la Universidad del Valle en Cali, Colombia. En 2011 se tituló como M. Sc. con énfasis en Electrónica en la misma universidad. Desde 2006 ha estado involucrada en la ejecución de proyectos relacionados con el procesamiento digital de señales. En la actualidad pertenece al grupo de Arquitecturas Digitales y Microelectrónica de la Escuela de Ingeniería Eléctrica y Electrónica de la Universidad del Valle y al grupo INTELIGO de la

Facultad de Ingeniería de la Institución Universitaria Antonio José Camacho donde enseña desde 2007.



Álvaro Bernal Noreña, en 1987 obtuvo su título de Ingeniero Eléctricista en la Universidad del Valle en Cali, Colombia. En 1992 se tituló como M. Sc. con énfasis en microelectrónica en la Universidad de São Paulo, Brasil. Obtuvo el doctorado con énfasis en microelectrónica en el Instituto Politécnico Nacional de Grenoble en Francia en 1999. Desde 1985, ha estado involucrado en la ejecución de proyectos relacionados con el diseño de CIs CMOS de bajo consumo y AsGa utilizados en aplicaciones de comunicaciones y criptografía. En la actualidad es

director del grupo de Arquitecturas Digitales y Microelectrónica y profesor titular de la Escuela de Ingeniería Eléctrica y Electrónica de la Universidad del Valle, donde enseña desde 1992.